DAQN: Deep Auto-encoder and Q-Network

Daiki Kimura
IBM Research AI
Email: daiki@jp.ibm.com

***Abstract*—The deep reinforcement learning method usually requires a large number of training images and executing actions to obtain sufficient results. When it is extended a real-task in the real environment with an actual robot, the method will be required more training images due to complexities or noises of the input images, and executing a lot of actions on the real robot also becomes a serious problem. Therefore, we propose an extended deep reinforcement learning method that is applied a generative model to initialize the network for reducing the number of training trials. In this paper, we used a deep q-network method as the deep reinforcement learning method and a deep auto-encoder as the generative model. We conducted experiments on three different tasks: a cart-pole game, an atari game, and a real-game with an actual robot. The proposed method trained efficiently on all tasks than the previous method, especially 2.5 times faster on a task with real environment images.**

I. Introduction

The deep reinforcement learning algorithms, such as deep q-network method [1], [2], deep deterministic policy gradients [3], and asynchronous advantage actor-critic [4], are the suitable methods for implementing interactive robot (agent). The method chooses an optimum action from a state input. These methods were often evaluated in a simulated environment. It is easy in the simulator to obtain input, to perform actions, and to get a reward. However, giving rewards to the real robot for each action needs human efforts. Also, performing training trials on the actual robot takes time and has risks of hurting the robot and environment. Thus, reducing the number of giving the reward and taking an action is necessary. Moreover, when we consider introducing into the real environment, state inputs will have a wider diversity. For example, if there is a state of "in front of a human", there are almost infinite variations of the visual representation of the human. Also, the input from an actual sensor contains complex background noises. Hence, a larger amount of training is essentially required than in the simulator environment.

On the other hand, a generative model has recently become popular to pre-train a neural network. An auto-encoder [5], [6] is one of the well-known generative model methods. Some studies, such as [7], [8], report the auto-encoder reduced the number of training steps for the classification task. We, therefore, assume pre-training helps to reduce the number of reinforcement learning steps. Moreover, it only requires inputs during pre-training; class labels of data are unnecessary. Hence, the training data of the pre-training doesn't need rewards for reinforcement learning; that means we can obtain these data from a random policy agent, or the environment around the robot without any action.

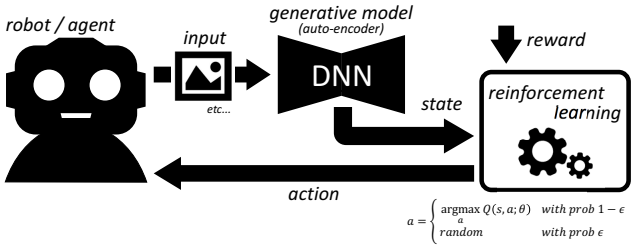

Fig. 1. Deep Auto-encoder and Q-Network

Therefore, we proposed an extended deep reinforcement learning method that is applied a generative model to initialize the network for reducing the number of training trials. In this paper, we used a deep q-network method [1], [2] as deep reinforcement learning, and a deep auto-encoder [6] as a generative model. We named this proposed method "deep auto-encoder and q-network" (daqn) in this paper. The overview is shown in Figure 1. This method first trains a network by the generative model with a random policy agent or inputs without actions. Then, the method trains policies by reinforcement learning which has the pre-trained network. The expected advantage is decreasing the number of training trials of the reinforcement learning phase. It is true that the proposed method additionally requires the training data for pre-training. However, the cost of obtaining this data is much lower than data for the reinforcement learning. This is because rewards and optimal actions are not required for the pre-training data. In this paper, we evaluate in three different environments, including a physical interaction with a real robot. Note that we focus on discrete actions in this paper for a clear discussion about contributions by a simple architecture. We assume the proposed method will be easily applied to continuous controls with such reinforcement learning methods [3], [4].

The contributions of this paper are to clarify the benefit of introducing the generative model into the deep reinforcement learning (especially, the auto-encoder into the deep q-network), conditions for its effectiveness, and a requirement of pre-training data. The most similar work is [9]. They copied an auto-encoder network to deep q-network in atari environment. However, they concluded the pre-training results show lower performance. The main reason are it used only first layer to copy and some training parameters are not proper. We use all layers and change the parameters. And we conduct an experiment in the real environment that is more complex than the atari; thus, a benefit of pre-training will be expected more.

II. Deep Auto-encoder and Q-Network

The proposed method has following steps: (1) trains a network by the deep auto-encoder, (2) deletes the decoder-layers, and adds a fully-connected layer on the top of encoder-layers, and (3) trains policies by the deep q-network algorithm.

The method first trains inputs by a deep auto-encoder for pre-training the network. The auto-encoder has encoder and decoder components, which can be defined as transitions ϕ and ψ. Here, we define $\mathcal{X} = \mathbb{R}^n, \mathcal{Y} = \mathbb{R}^m$. Let ϕ^* and ψ^* be trained by reconstructing its own inputs:

$$\phi : \mathcal{X} \to \mathcal{Y}, \ \ \psi : \mathcal{Y} \to \mathcal{X}, \ \ \boldsymbol{x} \in \mathcal{X} \tag{1}$$

$$\phi^*, \psi^* = \arg\min_{\phi,\psi} \|\boldsymbol{x} - (\psi \circ \phi)\boldsymbol{x}\|^2. \tag{2}$$

updating the feature encoder and decoder in the auto-encoder

When the input is a simple vector, the proposed method uses a one-dimensional auto-encoder [5]; when the input is an image, it uses a convolutional auto-encoder [6]. Importantly, the training data of this step will be obtained through a random policy of the agent, or the captured data from the environment without any action of the robot.

Next, the method removes decoder-layers of the auto-encoder network, and adds a fully-connected layer at the top of encoder-layers for discrete actions. Note that the weights of this added layer are initialized by random values.

Then, the method trains the policy by deep q-network [1], [2], which is initialized by the pre-trained network parameters from previous steps. The deep q-network is based on Q-learning algorithm [10]. The algorithm has an "action-value function" to calculate the quantity for a combination of state S and action A; $Q : S \times A \to \mathbb{R}$. Then, the update function of Q is,

$$Q(s_t, a_t) \leftarrow$$
$$Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right), \tag{3}$$

where the right-side $Q(s_t, a_t)$ is the current value, α is a learning ratio, r_{t+1} is a reward, γ is a discount factor, and $\max_a Q(s_{t+1}, a)$ is a maximum estimated action-value for state s_{t+1}.

Therefore, the loss function of the deep q-network is,

$$L(\theta) = \mathbb{E}_{(s_t, a_t, r_{t+1}, s_{t+1}) \sim \mathcal{D}} \left[(y - Q(s_t, a_t; \theta))^2 \right] \tag{4}$$

$$y = \begin{cases} r_{t+1} & \text{terminal} \\ r_{t+1} + \gamma \max_a Q(s_{t+1}, a; \theta^-) & \text{non-terminal,} \end{cases} \tag{5}$$

where θ is the parameters of deep q-network, $\mathcal{D}$ is an experience replay memory [11], $Q(s_t, a_t; \theta)$ will be calculated by the deep structure, and γ is a discount factor. θ^- is the weights that are updated fixed duration; this technique was also used in the original deep q-network method [1].

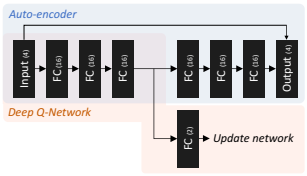

Fig. 2. Network for cart-pole game: blue part is auto-encoder component, orange part is deep q-network, and "FC" means a fully-connected layer.

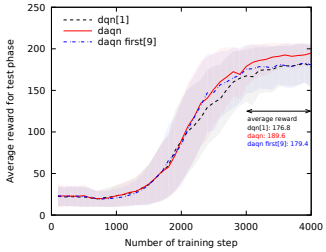

Fig. 3. Rewards for cart-pole game: the maximum reward is 300. Note that optimizer setting is only for auto-encoder (ae); the optimizer of dqn is same as dqn method.

III. Experiment

We conduct three different types of games in this study: a cart-pole game [12], an Atari game which is implemented in the arcade learning environment [13], and an interactive game on an actual robot in the real environment. The cart-pole is a simple game; hence, this is a base experiment. The Atari game, which is also used in the original deep q-network study or some related works, contains image inputs and a complex game rule. However, the images are taken from a simulated environment; specifically, images are from the game screen. The third environment is a real game. In this paper, we choose a "rock-paper-scissors" game, which is one of the various human hand-games with discrete actions. We adapt it to an interactive game between a real robot and a human. The input of this game is significantly complex than other experiments due to the real environment.

We used the OpenAI Gym framework [14] for simulating the cart-pole game and the Atari game. Also, we used Keras-library [15] and Keras-RL [16] for conducting the experiments.

A. Cart-pole game

1) Environment: The cart-pole game [12] is a game in which one pole is attached by a joint to a cart that moves along a friction-less track. The agent controls the cart to prevent the pole from falling over. It starts at the upright position, and it ends when the pole is more than 15 degrees from vertical.

In this game, the agent obtains four-dimensional values from a sensor and chooses an optimal action from two discrete actions: moving right or left. To conduct an experiment, we first design a simple network that has three hidden layers and final layer for the deep q-network (dqn). Figure 2 shows the details of the whole network of the proposed method. The activation functions of all full-connected layers are a ReLU function [17]. The proposed method first pre-trains data from a random agent by auto-encoder algorithm; this is the blue part in Figure 2. Next, the method adds a full-connected layer at top of the encoder-layers. Then, it starts to train policies by using the dqn algorithm; this is the orange part in Figure 2. We compare the training efficiency with the original dqn method that doesn't have pre-training. Note that "training step" in the

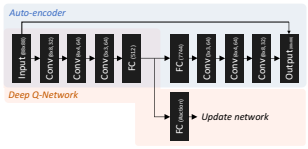

Fig. 4. Proposed network for Atari game: "Conv" means a convolutional layer.

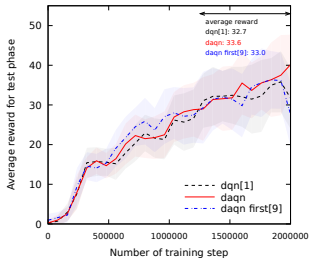

Fig. 5. Average rewards for Breakout.

i	ii	iii	iv	v
input	input	input	input	input
conv	conv	conv	conv	conv
max	max	max	max	max
conv	conv	conv	conv	conv
max	conv	conv	conv	conv
fc	fc	max	max	max
softmax	softmax	fc	conv	dropout
		softmax	max	conv
			fc	max
			softmax	dropout
				fc
				fc
				softmax
95.7%	97.2%	**97.8%**	97.6%	97.1%

TABLE I
NETWORKS AND ACCURACY FOR CLASSIFICATION TASK.

evaluation is for dqn component, excludes the number of auto-encoder training. Because the pre-training data is acquired from a random policy; thus it doesn't require much cost.

The pre-training data is inputs of 10 thousand (10k) steps from a random policy. The loss of auto-encoder is a mean square error function, the optimizer is an adaptive moment estimation (adam) [18], and we train 25 epochs. Also, we try the [9] method that uses only first layer of the pre-trained network to copy to dqn. However, we use new parameters that are same as our method; some are different from [9]. The optimizer of dqn component is adam, and exploration policy is Boltzmann approach. Note that all parameters and settings of the dqn component in daqn are same as those in the original dqn method.

2) Results: Figure 3 shows the reward curves for test phase with different numbers of training steps. Results were computed from running 100 episodes, 10 different training-trials; then the line is average of these results. The average rewards after 3k training steps were 176.8, **189.6**, and 179.4, for the dqn, the proposed method (daqn), and [9] which uses only first layer, respectively. These results show pre-training by auto-encoder was efficient to improve the reward.

B. Atari game

1) Environment: In the previous study [1], [2], they proposed a trainable deep convolutional neural network. We use the same network for this proposed method. However, we change the initial image size from 84×84 to 88×88, due to adding an auto-encoder component. All other parameters and settings were the same as those in the previous study.

We choose "Breakout" for evaluation; because it's one of the games in which the dqn trained successfully [1]. Figure 4 shows the proposed network. The pre-training data is 100k step images which are captured by a random policy agent. We use a fixed learning rate (0.01) with stochastic gradient descent for auto-encoder component, and train 25 epochs. Also, we try the [9] method which uses only first layer to copy; parameters are same as the proposed method, thus different from [9].

2) Results: Figure 5 shows the reward curves with different numbers of training steps. The statistics were computed from running 20 episodes. The average rewards after training 1.25 million steps were 32.7, **33.6**, and 33.0, for dqn, daqn, and [9], respectively. These results show the proposed method has the best result; however, it is similar to the dqn result.

C. Real-world game

1) Game setting: In this section, we conduct an experiment to evaluate the proposed method in the real environment. However, there is no suitable previous game-task with such environment. Hence, we choose a rock-paper-scissors game from some interactive human-games based on discrete actions. The optimal action of reinforcement learning is to beat a human-opponent by getting an opponent's hand image. The possible actions are to make a posture which represents 'rock', 'paper', and 'scissors'.

2) Pre-experiment: First, we must find a deep network that can do classification for these hand images. We take totally 50k images of four types of images: 'rock'-hand, 'paper'-hand, 'scissors'-hand, and background. During this shooting, we change a person, a hand (right or left hand), a height of the hand, wearing clothes, background environment, and a light condition. Figure 6 shows examples of taken images. Furthermore, these hand gesture images are not only the 'completed' figure but also the figure during the changing hand, such as a right-up image of the 'paper' in Figure 6. We prepare some networks to classify into four classes. The upper part of Table I describes the detail of networks. Images were initially cropped to a 120×120 color image. Each convolutional and fully-connected layer has the ReLU activation function. Training (90%) and test (10%) images were separated.

Figure 7 shows accuracy curve from five different trials, and the final row of Table I shows the average accuracy after 80 epochs. According to these results, network iii gave the highest accuracy, we will use this network structure.

3) Environment: Figure 8 shows the proposed method with network iii. In order to clear a requirement of pre-training data, we prepare the following three types of pre-training data: all images that are taken in pre-experiment (three hand-types and background), only background images, and images of handwritten digits (mnist) [19]. If the proposed method can train from only the background images, the cost to take the image is lower than hand images. Because we just require taking around the robot's environment; the human hand is not necessary. And, of course, if we can train from mnist, it is

Fig. 6. Sample images for rock-paper-scissors game

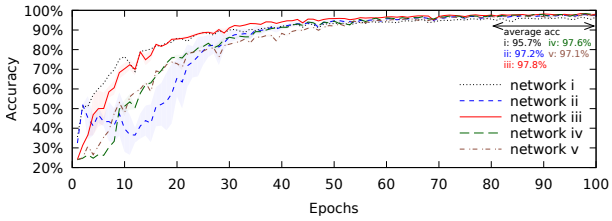

Fig. 7. Accuracy curve of classification

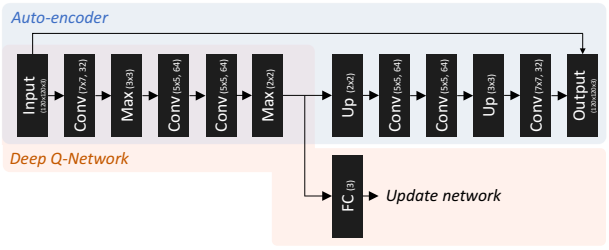

Fig. 8. Proposed network for real game: "up" is up-sampling.

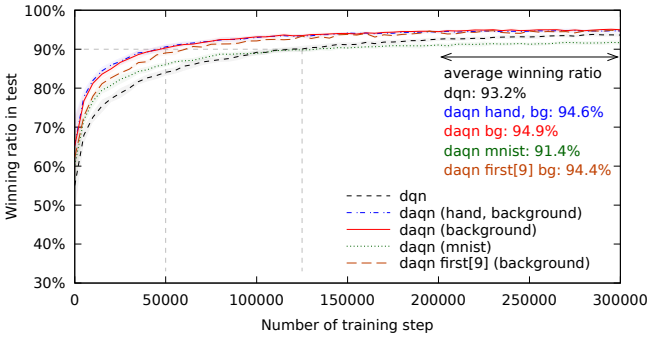

Fig. 9. Winning ratio curve for the rock-paper-scissors game

the lowest cost to prepare. We train 30 epochs for the auto-encoder component with an adam optimizer, random image shifting, rotating, and flipping.

After the auto-encoder training, it trains policies by dqn component. We use only hand-types images from taken images in the previous experiment; background images were excluded. The reward is $+1$, 0, or -1, when the agent wins, draws, or loses, respectively. The settings of dqn method and dqn component in daqn are as follows: learning ratio is 0.001 with sgd, size of replay memory is 1000, batch size is 32, exploration policy is ϵ-greedy, and loss is a mean square error function. And we compare with the [9] method; training parameters are same as daqn.

4) Results: Figure 9 shows a graph for the winning ratio curve of test images with different numbers of training steps. Each statistic was computed from 4k test-trials, and 5 different training-trials. The average winning ratios after 200k training are 93.2%, 94.6%, **94.9%**, 91.4%, and 94.4% for dqn, daqn which is pre-trained images of hand-types and background, daqn pre-trained background images, daqn pre-trained mnist images, and [9] method pre-trained background images, respectively. According to these results, the proposed method can train faster than the dqn method; for example, it is around 2.5 times faster to reach a 90% winning ratio. Also, the background images are enough to pre-train the network. Although mnist images initially help the improvement, the final winning ratio is lower than the dqn method. It means this pre-training data misled feature extraction in the reinforcement learning. Therefore, the requirement of pre-training data is these must have come from a domain related environment; however, backgrounds of the environment are accepted. The processing time for 300k training that includes evaluating was around 16.7 hours with Nvidia K40, and one test step took around 5.01 milli-seconds with Nvidia GTX Titan.

Moreover, we implement this game to an actual robot with the daqn pre-trained background. We attach a video that shows each robot reaction with a different number of training trials. In the video, the robot wins only one time at first. It can win gradually, it finally beats all types of opponent hands.

	Input complexity	Game complexity	DAQN
Cart-pole	Simple	Simple	$>$ DQN
Atari	Complex	Complex	$\simeq$ DQN
Real game	Highly complex	Simple	$\gg$ DQN

TABLE II
CONCLUSION OF THESE EXPERIMENTS

IV. Discussions

We applied the proposed method to different games: a simple game, a complex game in a simulator, and a real game. Table II shows the conclusion of these experiments. The proposed method has a pre-training component for the input; therefore, the complexity of the input directly affects the advantage of the proposed method. Hence, the most effective case is the game in the real environment. On the other hand, although the proposed method has a better result, the contribution is not so high in the atari game. We guess the main reason is a complexity of game. The pre-training data is based on the random policy; hence, images of the game did not change very much due to the difficulty of the game. Therefore, the pre-trained network parameters of daqn are easy to be acquired during initial steps of the dpn. Hence, the complexity of the game is also important for this pre-training.

Although it is less, the number of training trials of the proposed method is still large. It is still difficult to implement a system that will try "live training" on a physical robot. The reason for this long training is the difficulty to train inputs from the real environment. If we can use an improved deep reinforcement learning method, we hope it will converge faster; however, the processing time for test phase is also important for a real robot.

V. Related work

The deep q-network method [1], [2] is a method that successfully applies a deep convolutional neural network [20] to reinforcement learning [21] for training a policy of computer games. The convolutional neural network was originally inspired from neocognitron [22], then it has become a well-known approach for extracting high-level features from a raw image. This has especially led to significant breakthroughs in image processing studies [23], [24], [25].

Reinforcement learning [21] is an approach for training action policies with maximizing future cumulative rewards. There are several algorithms, such as Q-learning [10] and TD-gammon [26]. The TD-gammon applied a multi-layer perceptron for calculating approximated values, this achieved a human level to play a backgammon game.

The deep q-network (dqn) [1], [2] has a deep structure network that calculates the q-value of [10] from a raw input. The neural fitted q-learning (nfq) [27] also had a similar mechanism; however, the nfq used a batch update that has a computational cost per iteration. Also, the dqn used stochastic gradient updates that have a low constant cost per iteration [2]. Moreover, dqn has the latest convolutional neural network.

The deep auto-encoder was proposed for the dimensionality reduction [5]. However, it has also become more widely used for learning a generative model of the data [7], [8]. Also, the original deep auto-encoder [5] has a one-dimensional structure. However, it is difficult to preserve the spatial locality information of images with this structure. Therefore, a convolutional auto-encoder was proposed [6], and some studies applied to various problems [28], [29], [30].

A study to combine the spatial auto-encoder and reinforcement learning for real robot was proposed [31], they used the spatial auto-encoder for understanding the camera input from the robot. However, they used the spatial auto-encoder for describing the environment, such as the positions of objects, this is not for pre-training the network. Another reinforcement learning study with the auto-encoder was also proposed [32], they introduced the additional reward, which likes "curiosity" [33], by the auto-encoder. However, they used auto-encoder as a method for reducing the dimension of state input, which is same application example as the original deep auto-encoder [5].

The similar study with the proposed method is the deep auto-encoder neural networks in reinforcement learning [34]. This method first trains features by the auto-encoder, then trains policies by the batch-mode reinforcement learning algorithm. The architecture of this method is similar to ours; however, the motivation is different. They used the auto-encoder for reducing the dimension of input for the reinforcement learning. We used the auto-encoder to reduce the number of training trials for the reinforcement learning phase. Hence, we discussed the training efficiency in this paper. Furthermore, we used the latest convolutional auto-encoder and the latest deep reinforcement learning method.

The pre-training method with neural fitted q-learning was proposed [35]. They tried to pre-train from the completely random policy, and "hint-to-goal heuristic" policy. They reported pre-training without "hint" seems to do nothing. We believe the reason is the input complexities; they only tried the Mountain Car and Puddle World which are simple. In this paper, we discussed on the more complex environment.

The most similar study is [9]. They copied pre-trained network parameters by auto-encoder to deep q-network, and they evaluated on atari games. However, they concluded "the results generally show lower performance for cases with pre-training". We think this has three reasons: type of pre-trained data, structure of copied network, and hyper-parameters for training. First, they mainly focused on transferring trained network by multiple games or imagenet; they called "SSAE" and "INAE". These results are expected to become bad; this is same as the mnist result in this paper. Second, they only copied the pre-trained parameters of first layer; this is not so sufficient to help the dqn. They didn't try to copy "all" network of auto-encoder to dqn in "GSAE" case. Third, when we tuned the some hyper-parameters for training, the pre-training helped slightly even if we use their method on atari game. And the big difference with their paper is they only discussed atari; however, we conducted an experiment of the real game that is expected to have a good advantage of pre-training.

A robot to beat a human at rock-paper-scissors was proposed [36], they constructed a new active sensing system that can actively track and recognize the human hand by using a high-speed vision system. However, they didn't use the reinforcement learning, thus it could not train the optimal action. And they used a heuristic algorithm to understand the human hand, it is not robust in the noisy environment.

VI. Conclusion

We proposed an extended deep reinforcement learning that is applied one of the generative models to reduce the number of training steps. We evaluated it by three different games: a basic cart-pole game, a well-known atari game, and a "rock-paper-scissors" game in the real environment. Our method could train efficiently in all conditions; it especially works well when the input image has high complexity. For example, it trained 2.5 times faster than the original deep q-network method. Also, it could train from the background images which can be easily taken. We expect introducing the generative model must be required the deep reinforcement learning on actual robots in the real environment.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb 2015.

[2] X. Guo, S. Singh, H. Lee, R. L. Lewis, and X. Wang, "Deep learning for real-time atari game play using offline monte-carlo tree search planning," in *Advances in Neural Information Processing Systems 27*, Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2014, pp. 3338–3346.

[3] T. Lillicrap, J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *The International Conference on Learning Representations (ICLR)*, San Juan, Puerto Rico, 2016. [Online]. Available: http://arxiv.org/abs/1509.02971

[4] V. Mnih, A. Puigdomenech Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous Methods for Deep Reinforcement Learning," *International Conference on Machine Learning*, 2016.

[5] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[6] J. Masci, U. Meier, D. Cireşan, and J. Schmidhuber, "Stacked convolutional auto-encoders for hierarchical feature extraction," in *Proceedings of the 21th International Conference on Artificial Neural Networks - Volume Part I*, ser. ICANN'11. Berlin, Heidelberg: Springer-Verlag, 2011, pp. 52–59.

[7] D. Erhan, Y. Bengio, A. Courville, P.-A. Manzagol, P. Vincent, and S. Bengio, "Why does unsupervised pre-training help deep learning?" *J. Mach. Learn. Res.*, vol. 11, pp. 625–660, Mar. 2010. [Online]. Available: http://dl.acm.org/citation.cfm?id=1756006.1756025

[8] Q. V. Le, "Building high-level features using large scale unsupervised learning," in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, May 2013, pp. 8595–8598.

[9] T. Sandven, "Visual pretraining for deep q-learning," Master's thesis, Master's thesis, NTNU, 2016.

[10] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

[11] L.-J. Lin, "Reinforcement learning for robots using neural networks," *Technical report, DTIC Document*, 1993.

[12] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, Sept 1983.

[13] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Int. Res.*, vol. 47, no. 1, pp. 253–279, May 2013. [Online]. Available: http://dl.acm.org/citation.cfm?id=2566972.2566979

[14] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016. [Online]. Available: http://gym.openai.com/

[15] F. Chollet, "keras," https://github.com/fchollet/keras, 2015.

[16] M. Plappert, "keras-rl," https://github.com/matthiasplappert/keras-rl, 2016.

[17] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, J. Frnkranz and T. Joachims, Eds. Omnipress, 2010, pp. 807–814. [Online]. Available: http://www.icml2010.org/papers/432.pdf

[18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *The International Conference on Learning Representations (ICLR)*, vol. abs/1412.6980, USA, 2015. [Online]. Available: http://arxiv.org/abs/1412.6980

[19] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 1998.

[20] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[21] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press Cambridge, 1998, vol. 1.

[22] K. Fukushima, "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position," *Biological Cybernetics*, vol. 36, no. 4, pp. 193–202, 1980.

[23] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25*, F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2012, pp. 1097–1105.

[24] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015, pp. 1–9.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[26] G. Tesauro, "Temporal difference learning and td-gammon," *Commun. ACM*, vol. 38, no. 3, pp. 58–68, Mar. 1995.

[27] M. Riedmiller, "Neural fitted q iteration – first experiences with a data efficient neural reinforcement learning method," in *Proceedings of the 16th European Conference on Machine Learning*. Berlin, Heidelberg: Springer-Verlag, 2005, pp. 317–328.

[28] X. Mao, C. Shen, and Y. Yang, "Image restoration using very deep fully convolutional encoder-decoder networks with symmetric skip connections," in *Advances in Neural Information Processing Systems (NIPS'16)*, 2016.

[29] O. K. Oyedotun and K. Dimililer, "Pattern recognition: invariance learning in convolutional auto encoder network," *International Journal of Image, Graphics and Signal Processing*, vol. 8, no. 3, p. 19, 2016.

[30] V. Turchenko and A. Luczak, "Creation of a deep convolutional auto-encoder in caffe," *CoRR*, vol. abs/1512.01596, 2015. [Online]. Available: http://arxiv.org/abs/1512.01596

[31] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel, "Deep spatial autoencoders for visuomotor learning," in *International Conference on Robotics and Automation (ICRA)*, 2016.

[32] B. C. Stadie, S. Levine, and P. Abbeel, "Incentivizing exploration in reinforcement learning with deep predictive models," *CoRR*, vol. abs/1507.00814, 2015. [Online]. Available: http://arxiv.org/abs/1507.00814

[33] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *International Conference on Machine Learning (ICML)*, 2017.

[34] S. Lange and M. Riedmiller, "Deep auto-encoder neural networks in reinforcement learning," in *The 2010 International Joint Conference on Neural Networks (IJCNN)*, July 2010, pp. 1–8.

[35] F. Abtahi and I. Fasel, "Deep belief nets as function approximators for reinforcement learning," in *Proceedings of the 15th AAAI Conference on Lifelong Learning*, ser. AAAIWS'11-15. AAAI Press, 2011, pp. 2–7. [Online]. Available: http://dl.acm.org/citation.cfm?id=2908756.2908757

[36] K. Ito, T. Sueishi, Y. Yamakawa, and M. Ishikawa, "Tracking and recognition of a human hand in dynamic motion for janken (rock-paper-scissors) robot," in *2016 IEEE International Conference on Automation Science and Engineering (CASE)*, Aug 2016, pp. 891–896.